



### 0.0.1 Quant Simulation Challenge 1: Price Making

In this challenge, your task is to build a simple, automated market maker. Using **object-oriented programming**, you will create a class that can provide a two-sided price quote (a bid and an offer) for any given asset.

Your model will need to calculate quotes that are **2% away** from the current market price.

What is Market Making?

When a large client (like a hedge fund) wants to trade, they ask a Market Maker for a price.

The client tells you what they want to trade (e.g., Apple shares) and how many.

They don't tell you if they want to buy or sell.

Because of this, you must provide a price for both scenarios:

Bid Price: The price the client can sell at.

Offer Price: The price the client can buy at.

**Calculation Rules** For this challenge, your bid and offer prices should be calculated using a simple 2% spread around the **reference price** (the current market price).

- **Bid Price** = Reference Price  $\times (1 - 0.02)$
- **Offer Price** = Reference Price  $\times (1 + 0.02)$

For Example:

If a client requests a quote for Apple and its current reference price is \$100, your market maker would provide the following quote:

Bid: \$98.00 — \$102.00 :Offer

Now, let's import the packages you will need for this notebook.

```
[1]: # The code in this cell is used to import the packages to be used throughout
      ↪ this notebook.
      # The following are private packages available only during this simulation:
      from AmplifyQuantTrading import Data
      from AmplifyQuantTrading import Exchange
      from AmplifyQuantTrading import MarketMaker
      from AmplifyQuantTrading import HedgeFund as hf
      # The following are publicly available packages:
```

```
from matplotlib import pyplot as plt
from pandas import *
```

```
[2]: # HIDDEN PARAMS BOX
```

Setting Up Our Data

The code below assigns our data to two main variables that we'll use throughout the project:

prices: A series of live market prices for our assets.

price\_requests: A series of trade requests from our clients.

```
[3]: prices = Data.get_price_series("PricestoFeedserver")
price_requests = Data.get_price_requests("PriceRequeststoFeedserver")
```

### 0.0.2 Task A: Create a Sample Dataset

To make it easier to build and test your code, your first task is to create a smaller sample of data. You'll select the first **ten** trade requests from the main `price_requests` list.

Your Task

Iterate through the `price_requests` data and append the first 10 requests to a new list called `test_requests`.

Check your work: The final `test_requests` list should contain exactly 10 items.

```
[4]: test_requests = []
```

```
[5]: for index in range(0, 10): # Select the first ten prices in the data.
      # Insert your answer code here
      test_requests.append(price_requests[index])
```

<h4>Output Check</h4>

<p>Run the cell below to display your final <code>test\_requests</code> list.</p>

<p>You should see a list containing ten requests you selected. Each item in the list follows the format: <code>[ ticker, date, volume ]</code></p>

<p><code>[ ticker, date, volume ]</code></p>

```
[6]: print(test_requests)
```

```
[['FB', 0, 3010], ['AAPL', 10000, 6929], ['FB', 30000, 7015], ['FB', 70000, 4910], ['AAPL', 105000, 5783], ['AAPL', 110000, 3280], ['FB', 115000, 2465], ['AAPL', 120000, 656], ['FB', 130000, 1948], ['FB', 150000, 1386]]
```

<h4>For Grading Purposes Only</h4>

<p>You don't need to change anything in the cell below. It's used to check your work automatically.</p>

```
[7]: # GRADING CELL
```

### 0.0.3 Task B: Find the Reference Price for Each Request

Now that you have your `test_requests`, you need to find the correct market price for each one. This is the **reference price** you'll use later to calculate your bid and offer quotes.

Your Task

Create an empty list called `request_with_prices`.

Loop through each request in your `test_requests` list.

For each request, find the matching price from the prices data by matching the ticker and date.

Append the original request and its reference price to your `request_with_prices` list.

<h4>Required Output Format</h4>

<p>Your final list must be a list of tuples. Each tuple should contain the original request (w

<p><b>Structure:</b> <code>[ ( [ticker, date, volume], reference\_price ), ... ]</code></p>

```
[8]: request_with_prices = []
```

```
[10]: for price in prices: # Iterate through all the prices to match them against
    the price requests.
        for request in test_requests: # Iterate through the requests from the
    Hedge Funds.
            # Insert your answer code here
            ticker, date, volume = request
            if price[0] == ticker and price[1] == date:
                request_with_prices.append((request, price[2]))
            break
```

<h4>Output Check</h4>

<p>Run the cell below to check your <code>request\_with\_prices</code> list.</p>

<p>The output confirms each trade request has been paired with its reference price, following t

<p><b>Structure:</b> <code>[ ( [ticker, date, volume], reference\_price ), ... ]</code></p>

```
[11]: print(request_with_prices)
```

```
[(['FB', 0, 3010], 194.44), (['AAPL', 10000, 6929], 74.63), (['FB', 30000,
7015], 195.75), (['FB', 70000, 4910], 193.3), (['AAPL', 105000, 5783], 73.13),
(['AAPL', 110000, 3280], 73.83), (['FB', 115000, 2465], 185.76), (['AAPL',
120000, 656], 76.56), (['FB', 130000, 1948], 191.32), (['FB', 150000, 1386],
195.82)]
```

<h4>For Grading Purposes Only</h4>

<p>You don't need to change anything in the cell below. It's used to check your work automatic

```
[12]: # GRADING CELL
```

### 0.0.4 Task C: Build a QuotedTrade Class

It's time to turn your data into a structured quote. Instead of using lists and tuples, you will now use **Object-Oriented Programming (OOP)** to create a `QuotedTrade` object for each request.

Using a class is a clean and professional way to package all the information related to a single trade. This makes your code easier to read, manage, and eventually send to clients.

Class Blueprint: `QuotedTrade`

Your class will act as a blueprint for every quote you create. Each object made from this class must contain the following attributes:

ticker: The asset's name (String)

trade\_volume: The number of shares requested (Integer)

ref\_price: The market price at the time of the request (Float)

bid\_price: Your calculated price for the client to sell at (Float)

offer\_price: Your calculated price for the client to buy at (Float)

date: The timestamp of the request (Integer)

Your Task

Use the defined `QuotedTrade` class.

Inside the loop, create `QuotedTrade` objects with the correct Bid and Offer Price.

```
[13]: class QuotedTrade:
    def __init__(self, ticker, trade_volume, ref_price, bid_price, offer_price,
                 date):
        self.ticker = ticker
        self.trade_volume = trade_volume
        self.ref_price = ref_price
        self.bid_price = bid_price
        self.offer_price = offer_price
        self.date = date

    def __str__(self):
        return f'Trade Request for {self.ticker}, {self.trade_volume} shares @_{self.ref_price} on {self.date}. Bid Price: {self.bid_price} and Offer Price: {self.offer_price}'

    def __repr__(self):
        return f'QuotedTrade(ticker={self.ticker}, trade_volume={self.trade_volume}, ref_price={self.ref_price}, bid_price={self.bid_price}, offer_price={self.offer_price}, date={self.date})'
```

<h4>Key Calculation: Bid & Offer Prices</h4>

<p>You must calculate the <code>bid\_price</code> and <code>offer\_price</code>. These should be

```
[14]: quoted_trades = []
```

```
[15]: # Replace the ? in this task with your answer.
```

```

for matched in request_with_prices:
    bid_price = matched[1] * (1 - 0.02) # Create the bid spread

    offer_price = matched[1] * (1 + 0.02) # Create the offer spread

    # Now create the QuotedTrade object and append it to the quoted_trades list.
    # Insert your answer code here
    quoted_trades.append(
        QuotedTrade(
            matched[0][0],    # ticker
            matched[0][2],    # trade_volume
            matched[1],        # ref_price
            bid_price,
            offer_price,
            matched[0][1]      # date
        )
    )

```

<h4>Output Check</h4>

<p>Run the cell below to display your final <code>quoted\_trades</code> list.</p>

<p>You should see a list containing ten <code>QuotedTrade</code> objects, confirming that your

[16]: `print(quoted_trades)`

```

[QuotedTrade(ticker=FB, trade_volume=3010, ref_price=194.44, bid_price=190.5512,
offer_price=198.3288, date=0), QuotedTrade(ticker=AAPL, trade_volume=6929,
ref_price=74.63, bid_price=73.1374, offer_price=76.12259999999999, date=10000),
QuotedTrade(ticker=FB, trade_volume=7015, ref_price=195.75, bid_price=191.835,
offer_price=199.665, date=30000), QuotedTrade(ticker=FB, trade_volume=4910,
ref_price=193.3, bid_price=189.434, offer_price=197.16600000000003, date=70000),
QuotedTrade(ticker=AAPL, trade_volume=5783, ref_price=73.13, bid_price=71.6674,
offer_price=74.59259999999999, date=105000), QuotedTrade(ticker=AAPL,
trade_volume=3280, ref_price=73.83, bid_price=72.3534, offer_price=75.3066,
date=110000), QuotedTrade(ticker=FB, trade_volume=2465, ref_price=185.76,
bid_price=182.04479999999998, offer_price=189.4752, date=115000),
QuotedTrade(ticker=AAPL, trade_volume=656, ref_price=76.56, bid_price=75.0288,
offer_price=78.0912, date=120000), QuotedTrade(ticker=FB, trade_volume=1948,
ref_price=191.32, bid_price=187.4936, offer_price=195.1464, date=130000),
QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000)]

```

<h4>For Grading Purposes Only</h4>

<p>You don't need to change anything in the cell below. It's used to check your work automatic

[17]: `# GRADING CELL`

### 0.0.5 Task D: Get the Hedge Fund's Decision

This is the final step of the process. You will now send each of your `QuotedTrade` objects to the hedge fund client (represented by the `hf` object) to get their decision. The client will respond by telling you if they want to **Buy** from you, **Sell** to you, or **Refuse** the trade entirely.

**Method Documentation:** `hf.show()`

This function simulates sending your quote to the hedge fund for a decision.

---

**Method:** `hf.show(trade)`

**Parameters:**

- `trade`: A single `QuotedTrade` object you created in the previous task.

**Returns:**

- An `HfResponse` object. This object contains the fund's action ('Buy', 'Sell', 'Refuse').

Your Task

Create an empty list called `hf_responses`.

Loop through your list of `quoted_trades` objects.

For each `QuotedTrade` object, pass it to the `hf.show()` method.

Append the resulting `HfResponse` object to your `hf_responses` list.

```
[18]: hf_responses = []

[19]: # Replace the ? in this task with your answer.

for trade in quoted_trades:
    response = hf.show(trade) # Assign the response to the return from the hf.
    # Insert your answer code here
    hf_responses.append(response)
```

**Output Check**

Run the cell below to see the final `hf_responses` list.

This output shows the decision for each of your quotes, completing the market-making cycle.

```
[20]: print(hf_responses)

[HfResponse(ticker=FB, trade_volume=3010, trade_price=190.5512, hf_action=sell,
ref_price=194.44, bid_price=190.5512, offer_price=198.3288, date=0),
HfResponse(ticker=AAPL, trade_volume=6929, trade_price=73.1374, hf_action=sell,
ref_price=74.63, bid_price=73.1374, offer_price=76.12259999999999, date=10000),
HfResponse(ticker=FB, trade_volume=7015, trade_price=199.665, hf_action=buy,
ref_price=195.75, bid_price=191.835, offer_price=199.665, date=30000),
HfResponse(ticker=FB, trade_volume=4910, trade_price=197.16600000000003,
hf_action=buy, ref_price=193.3, bid_price=189.434,
offer_price=197.16600000000003, date=70000), HfResponse(ticker=AAPL,
trade_volume=5783, trade_price=74.59259999999999, hf_action=buy,
ref_price=73.13, bid_price=71.6674, offer_price=74.59259999999999, date=105000),
```

```
HfResponse(ticker=AAPL, trade_volume=3280, trade_price=75.3066, hf_action=buy,
ref_price=73.83, bid_price=72.3534, offer_price=75.3066, date=110000),
HfResponse(ticker=FB, trade_volume=2465, trade_price=182.04479999999998,
hf_action=sell, ref_price=185.76, bid_price=182.04479999999998,
offer_price=189.4752, date=115000), HfResponse(ticker=AAPL, trade_volume=656,
trade_price=75.0288, hf_action=sell, ref_price=76.56, bid_price=75.0288,
offer_price=78.0912, date=120000), HfResponse(ticker=FB, trade_volume=1948,
trade_price=195.1464, hf_action=buy, ref_price=191.32, bid_price=187.4936,
offer_price=195.1464, date=130000), HfResponse(ticker=FB, trade_volume=1386,
trade_price=191.90359999999998, hf_action=sell, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000)]
```

<h4>For Grading Purposes Only</h4>

<p>You don't need to change anything in the cell below. It's used to check your work automatic

[21]: # GRADING CELL

## 0.0.6 Task E: Initialize the Market Maker

Now it's time to bring everything together. You will initialize the main `MarketMaker` class, which will act as the central controller for tracking your positions, quotes, and completed trades throughout the simulation.

<h4>The `MarketMaker` Object</h4>

<p>The `MarketMaker` object is the central controller for our simulation and is initialized with

- <li><code>current\_positions</code>: A dictionary tracking the volume of each asset you currently hold.
  - <li><em>Structure: <code>{ 'TICKER': position\_object, ... }</code></em></li></ul>
 </li>
  - <li><code>quoted\_trades</code>: A list to store all the <code>QuotedTrade</code> objects you have received.
  - <li><code>completed\_trades</code>: A list to store all trades that clients have accepted (or rejected).
  - <li><code>ETF\_positions</code>: A list used specifically to track your ETF trades.</li>
 </ul>

<h4>Your Task: Initialize the Object</h4>

<p>Run the code cell below to create your instance of the <code>MarketMaker</code> class. This

[22]: mm = MarketMaker.mm()

## 0.0.7 Task F: Log Your Quoted Trades

A crucial part of market making is keeping a detailed log of all activity. Your next task is to log every quote you've prepared by adding each one to your `market_maker` object.

<h4>Method: <code>market\_maker.add\_quoted\_trade()</code></h4>

<p>This method adds a single <code>QuotedTrade</code> object to the market maker's internal log

---

<p><b>Parameters:</b></p>

- <li><code>trade</code>: A single <code>QuotedTrade</code> object.</li></ul>

<p><b>Returns:</b></p>



<ul><li>A confirmation message (String) indicating if the trade was added successfully.</li></ul>

Your Task

Loop through the `quoted_trades` list you created earlier.

Inside the loop, call the `market_maker.add_quoted_trade()` method for each trade object to log it.

```
[23]: for quote in quoted_trades:
      # Use the function to add each quote to the Market Maker
      # Insert your answer code here
      mm.add_quoted_trade(trade)
```

<h4>Output Check</h4>

<p>Run the cell below to check the contents of the <code>market\_maker.quoted\_trades</code> list</p>

<p>This confirms that all ten of your quoted trades have been successfully logged inside the market maker</p>

```
[24]: print(mm.quoted_trades)

[QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000),
QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000),
QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000),
QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000),
QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000),
QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000),
QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000),
QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000),
QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000),
QuotedTrade(ticker=FB, trade_volume=1386, ref_price=195.82,
bid_price=191.90359999999998, offer_price=199.7364, date=150000)]
```

## 0.0.8 Task G: Create a CompletedTrade Class

When a hedge fund accepts one of your quotes, the trade is considered “completed.” To keep a clean record of your successful transactions, you will create a new `CompletedTrade` class.

This class will act as a blueprint for logging every trade that actually gets executed.

<h4>Class Blueprint: `CompletedTrade`</h4>

<p>Each object made from this class will represent a single successful trade and must contain the following attributes</p><ul>

<li><code>ticker</code>: The asset's name (String)</li>



- <li><code>trade\_volume</code>: The number of shares traded (Integer)</li>
- <li><code>trade\_price</code>: The final price the trade was executed at (Float)</li>
- <li><code>mm\_action</code>: The market maker's side of the trade ('Buy' or 'Sell') (String)</li>
- <li><code>ref\_price</code>: The original reference price (Float)</li>
- <li><code>bid\_price</code>: The original bid price you quoted (Float)</li>
- <li><code>offer\_price</code>: The original offer price you quoted (Float)</li>
- <li><code>date</code>: The timestamp of the request (Integer)</li>

Your Task

Define the CompletedTrade class.

Inside the class, create an **init** method.

The **init** method should accept all the necessary information from a hedge fund's response and set the attributes listed above.

```
[25]: class CompletedTrade:
    # Insert your answer code here
    def __init__(self, ticker, trade_volume, trade_price,
                 mm_action, ref_price, bid_price, offer_price, date):
        self.ticker = ticker
        self.trade_volume = int(trade_volume)
        self.trade_price = float(trade_price)
        self.mm_action = mm_action
        self.ref_price = float(ref_price)
        self.bid_price = float(bid_price)
        self.offer_price = float(offer_price)
        self.date = int(date)
```

### 0.0.9 Log Completed Trades & Update Positions

Now that you have the hedge fund's decisions, it's time to update your records. You'll process the `hf_responses` list, ignoring any refused trades and logging the successful ones. This final step will also update your inventory to reflect your new positions.

<h4> Method: <code>market\_maker.add\_trade()</code></h4>

<p>This crucial method logs a completed trade to your history and automatically updates your co

---

<p><b>Parameters:</b></p>

<ul><li><code>trade</code>: A single <code>CompletedTrade</code> object.</li></ul>

<p><b>Returns:</b></p>

<ul><li>A confirmation message (String) indicating if the trade was added successfully.</li></ul>

<h4> Important Logic</h4>

<p>You must only process responses where the trade was accepted. Check the <code>hf\_action</code>

<ul>

<li>If the action is 'Buy' or 'Sell', you should log it.</li>

<li>If the action is 'Refuse', you should ignore it and move to the next response.</li>

</ul>

Your Task

Loop through the hf\_responses list.

For each response, check if the hf\_action is not 'Refuse'.

If the trade was accepted, create an instance of your CompletedTrade class using the data from the response object.

Pass this new CompletedTrade object to the market\_maker.add\_trade() method to log it.

```
[26]: for response in hf_responses: # Loops through the hf_responses for each response

    # Collection of conditionals for "buy", "sell" and "refuse"
    if response.hf_action == "buy": # Identifies the HF action to be converted
        ↪into the MM action
        # Insert your answer code here
        completed_trade = CompletedTrade(
            ticker=response.ticker,
            trade_volume=response.trade_volume,
            trade_price=response.trade_price,
            mm_action=response.hf_action,
            ref_price=response.ref_price,
            bid_price=response.bid_price,
            offer_price=response.offer_price,
            date=response.date
        )
        mm.add_trade(completed_trade)

    elif response.hf_action == "sell":
        completed_trade = CompletedTrade(
            ticker=response.ticker,
            trade_volume=response.trade_volume,
            trade_price=response.trade_price,
            mm_action=response.hf_action,
            ref_price=response.ref_price,
            bid_price=response.bid_price,
            offer_price=response.offer_price,
            date=response.date
        )
        mm.add_trade(completed_trade)
```

<h4>Output Check</h4>

<p>Run the cell below to view the contents of your <code>market\_maker.completed\_trades</code> .  
<p>This output confirms your final log, which should only contain the trades that were success:

```
[27]: print(mm.completed_trades)
```

```
[<__main__.CompletedTrade object at 0x7f8cdf767410>, <__main__.CompletedTrade
object at 0x7f8cdfabb710>, <__main__.CompletedTrade object at 0x7f8cdf7666c0>,
<__main__.CompletedTrade object at 0x7f8cdf767800>, <__main__.CompletedTrade
```

```
object at 0x7f8cdf767b60>, <__main__.CompletedTrade object at 0x7f8cdf767bc0>,  
<__main__.CompletedTrade object at 0x7f8cdf766660>, <__main__.CompletedTrade  
object at 0x7f8cdf766c60>, <__main__.CompletedTrade object at 0x7f8cdf767bf0>,  
<__main__.CompletedTrade object at 0x7f8cdf767950>]
```

<h4>For Grading Purposes Only</h4>

<p>You don't need to change anything in the cell below. It's used to check your work automatically.</p>

[28]: `# GRADING CELL`

### 0.0.10 Task I: Visualize Your AAPL Quotes

A key skill for any quant is the ability to visualize data. You will now create a chart to see how your quoted bid and offer prices for **AAPL** behaved around the market's reference price over time.

<h4>Chart Requirements</h4>

<p>Your final plot must meet the following criteria:</p>

<ul>

<li><strong>Library:</strong> Use the <code>matplotlib</code> library.</li>

<li><strong>Data:</strong> Only include trades for the ticker <strong>'AAPL'</strong>.</li>

<li><strong>X-axis:</strong> Date</li>

<li><strong>Y-axis:</strong> Bid Price, Offer Price, and Reference Price.</li>

</ul>

Your Task

Filter the Data: Create new lists to hold the date, bid\_price, offer\_price, and ref\_price for AAPL trades only.

```
[29]: # Initialises the lists to store all the data  
bid_data = []  
offer_data = []  
quote_dates = []  
  
for trade in mm.completed_trades: # Iterate through the completed trades to  
    find the AAPL trades and store the data.  
    # Append the bid_data and offer_data and quote_dates.  
    # Insert your answer code here  
    if trade.ticker == "AAPL":  
        bid_data.append(trade.bid_price)  
        offer_data.append(trade.offer_price)  
        quote_dates.append(trade.date)
```

<h4>Output Check</h4>

<p>Run the cell below to print the lists you just created for your plot.</p>

<p>Before plotting, it's crucial to confirm that all lists have the <b>same length</b>. If they

```
[30]: print("bid_data:", bid_data)  
print("offer_data:", offer_data)  
print("quote_dates:", quote_dates)
```

```
bid_data: [73.1374, 71.6674, 72.3534, 75.0288]
offer_data: [76.12259999999999, 74.59259999999999, 75.3066, 78.0912]
quote_dates: [10000, 105000, 110000, 120000]
```

The quote\_dates, bid\_data, and offer\_data lists are now ready.

#### Your Task

The last piece of data you need for your chart is the reference price for each of your AAPL

- Create a new empty list called `ref_price_data`.
- You will need to find the reference price that corresponds to each date in your `quote_dates`.
- Loop through your data source (e.g., `request_with_prices` or `market_data`).

Check your work: The new `ref_price_data` list must have the same length as your other lists.

```
[31]: # Replace the ? in this task with your answer.

ref_data = []
ref_dates = []

for price in prices: # Iterate through the quotes to find each AAPL ticker and
    # matching price data.
    # Add filtering to ensure we find AAPL trades and all trades upto the last
    # quote_dates.
    # Append the ref_data and ref_dates
    # Insert your answer code here
    ticker = price[0]
    date = price[1]
    ref_price = price[2]

    if ticker == "AAPL" and date in quote_dates:
        ref_data.append(ref_price)
        ref_dates.append(date)
```

#### Output Check

Run the cell below to verify your lists before plotting.

Confirm that your new `ref_price_data` list has the **same length** as the `quote_dates`.

```
[32]: print("ref_data:", ref_data)
      print("ref_dates:", ref_dates)
```

```
ref_data: [74.63, 73.13, 73.83, 76.56]
ref_dates: [10000, 105000, 110000, 120000]
```

#### Your Task

All your data is now prepared. It's time to create the final visualization.

- Use `axes.plot()` to plot your **bid**, **offer**
- Customize your chart with a title, axis labels (e.g., 'Date', 'Price'), and a legend to

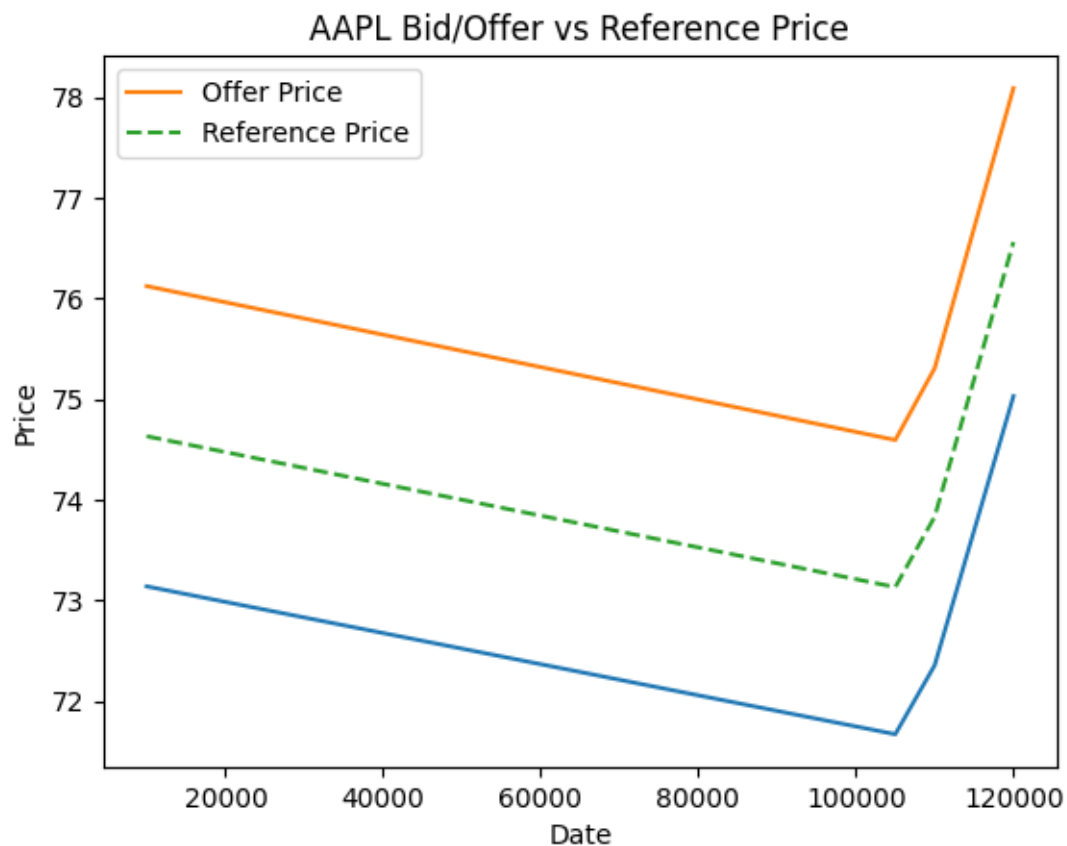
</ol>

```
[33]: axes = plt.subplot() # Creates the Axis.

axes.plot(quote_dates, bid_data) # Plot the X and Y axis using the quote_dates_
    ↪and bid_data.

# Plot the refData and offerData next
# Insert your answer code here
axes.plot(quote_dates, offer_data, label="Offer Price")
axes.plot(ref_dates, ref_data, label="Reference Price", linestyle="--")

axes.set_title("AAPL Bid/Offer vs Reference Price")
axes.set_xlabel("Date")
axes.set_ylabel("Price")
axes.legend()
plt.show()
```



<h4>For Grading Purposes Only</h4>

<p>You don't need to change anything in the cell below. It's used to check your work automatically.</p>

```
[ ]: # GRADING CELL
```

## 0.1 Challenge 2: Price Skewing & Risk Management

In this challenge, you'll build on the foundation of Challenge 1 by adding a new layer of real-world complexity: **price skewing**. In practice, a market maker doesn't always quote prices symmetrically around a reference price. Instead, they adjust their quotes to manage the risk of their current inventory.

**The Logic of Skewing**

The goal of skewing is to influence which side of your quote a client is more likely to take.

- If you are **long** an asset (you own too much), you want to encourage clients to sell.
- If you are **short** an asset (you owe it), you want to encourage clients to buy.

**For Example:**

If the reference price for Apple is **\$100**:

- When Long** (you want to sell): Your quote might be **Bid \$95 - \$101 Offer**. Both prices are shifted down to make selling more likely.
- When Short** (you want to buy): Your quote might be **Bid \$99 - \$105 Offer**. Both prices are shifted up to make buying more likely.

Let's add this layer of complexity to our code. First, we'll import the necessary packages.

```
[34]: # The code in this cell is used to import the packages to be used throughout
      ↪ this notebook.

      # The following are private packages available only during this simulation:
      from AmplifyQuantTrading import Data
      from AmplifyQuantTrading import Exchange
      from AmplifyQuantTrading import MarketMaker
      from AmplifyQuantTrading import HedgeFund as hf
      # The following are publicly available packages:
      from matplotlib import pyplot as plt
      from pandas import *
```

```
[35]: # HIDDEN PARAMS BOX
```

## 0.2 Initial Setup of Classes

Before we begin, let's define the core data structures we'll use to represent our trades.

**Class Blueprint: `QuotedTrade`**

This class acts as a container for every quote you send to a client. It must have the following attributes:

- `ticker`: String
- `trade_volume`: Integer
- `ref_price`: Float

```

    <li><code>bid_price</code>: Float</li>
    <li><code>offer_price</code>: Float</li>
    <li><code>date</code>: Integer</li>
</ul>

<h4>Class Blueprint: `CompletedTrade`</h4>
<p>This class is used to log the details of a successful trade after a client accepts your quote</p>
<ul>
    <li><code>ticker</code>: String</li>
    <li><code>trade_volume</code>: Integer</li>
    <li><code>trade_price</code>: Float</li>
    <li><code>mm_action</code>: String ('Buy' or 'Sell')</li>
    <li><code>date</code>: Integer</li>
</ul>

```

Your Task

In the code cell below, define both the QuotedTrade and CompletedTrade classes. Each class should have an **init** method that sets the attributes listed in its blueprint.

```

[36]: class QuotedTrade:
    def __init__(self, ticker, trade_volume, ref_price, bid_price, offer_price,
    ↪date):
        self.ticker = ticker
        self.trade_volume = trade_volume
        self.ref_price = ref_price
        self.bid_price = bid_price
        self.offer_price = offer_price
        self.date = date

class CompletedTrade:
    def __init__(self, ticker, trade_volume, trade_price, mm_action, ref_price,
    ↪bid_price, offer_price, date):
        self.ticker = ticker
        self.trade_volume = trade_volume
        self.trade_price = trade_price
        self.mm_action = mm_action
        self.ref_price = ref_price
        self.bid_price = bid_price
        self.offer_price = offer_price
        self.date = date

```

### 0.2.1 Challenge 2 - Task A: Implement a Risk-Based Skewing Algorithm

This task combines everything you've learned into a single, complete algorithm. You will process all incoming price requests, but this time, you'll skew your quotes based on your current inventory risk before sending them to the client and logging the results.

<h4>Key Functions for this Task</h4>

<p>You will need to use the following methods from your objects:</p>



```

<p><b>Market Maker (`market_maker`):</b></p>
<ul>
  <li><code>.current_positions</code>: To check your inventory for a given ticker.</li>
  <li><code>.add_quoted_trade()</code>: To log every quote you create.</li>
  <li><code>.add_trade()</code>: To log every completed trade and update your positions.</li>
</ul>
<p><b>Hedge Fund (`hf`):</b></p>
<ul>
  <li><code>.show()</code>: To send your quote to the client and get their response.</li>
</ul>

```

<h4>Remember to Log Everything!</h4>

<p>A key part of risk management is maintaining a complete audit trail. For every request, you

Your Task

Your goal is to create a single loop that processes every item in the price\_requests list. For each request, you must perform the following steps in order:

Check Your Risk: Before calculating a price, check your current position for that ticker in market\_maker.current\_positions.

Calculate Skewed Prices: Based on your position (long, short, or neutral), calculate the skewed bid and offer prices.

Create a Quote: Instantiate a QuotedTrade object with your skewed prices.

Log the Quote: Add the new QuotedTrade object to your logs using market\_maker.add\_quoted\_trade().

Show Quote to Client: Send your quote to the client using hf.show() to get their decision.

Process the Response: Check the HfResponse. If the trade was not 'Refuse'd:

Create an instance of your CompletedTrade class.

Log the successful transaction using market\_maker.add\_trade().

</li>

```

[37]: # The code below creates the MarketMaker Object
mm = MarketMaker.mm()
# The code below will assign the prices data series and price_requests data_
↪series to the two variables.
prices = Data.get_price_series("PricestoFeedserverExtended")
price_requests = Data.get_price_requests("PriceRequeststoFeedserverExtended")

```

```

[38]: # Insert your answer code here
ref_by_key = { (t, d): r for (t, d, r) in prices }

base_spread_pct = 0.02
skew_abs_pct    = 0.01

```

```

for request in price_requests:
    if len(request) == 3:
        ticker, trade_volume, date = request
    elif len(request) == 2:
        ticker, date = request
        trade_volume = 0
    else:
        continue

    ref_price = ref_by_key.get((ticker, date))
    if ref_price is None:
        continue

    # check risk
    pos_obj = mm.current_positions.get(ticker, 0)
    pos_val = getattr(pos_obj, "volume", pos_obj)
    if isinstance(pos_val, (int, float)):
        if pos_val > 0: # long
            skew = -skew_abs_pct
        elif pos_val < 0: # short
            skew = +skew_abs_pct
        else:
            skew = 0.0
    else:
        skew = 0.0

    # calculate skewed prices
    bid_price = ref_price * (1 - base_spread_pct + skew)
    offer_price = ref_price * (1 + base_spread_pct + skew)

    # create and log the quote
    quote = QuotedTrade(
        ticker=ticker,
        trade_volume=trade_volume,
        ref_price=ref_price,
        bid_price=bid_price,
        offer_price=offer_price,
        date=int(date)
    )
    mm.add_quoted_trade(quote)

    # show quote to client and process the response
    response = hf.show(quote)
    if str(response.hf_action).lower() in ("buy", "sell"):
        completed = CompletedTrade(
            ticker=response.ticker,
            trade_volume=response.trade_volume,

```

```

        trade_price=response.trade_price,
        mm_action=response.hf_action,
        ref_price=response.ref_price,
        bid_price=response.bid_price,
        offer_price=response.offer_price,
        date=response.date
    )
    mm.add_trade(completed)

```

[39]: *# GRADING CELL*

### 0.3 Challenge 2 - Task B: Visualize Your Skewed Quotes

The final task is to visualize your new pricing logic. By plotting your skewed quotes, you can clearly see how your algorithm adjusted its prices based on its trading risk throughout the simulation.

**Chart Requirements**

Your plot should show the relationship between your quotes and the market price for a single ticker.

- Ticker:** Choose one ticker from your logs to visualize.
- Data Source:** Use the final `market_maker.quoted_trades` list.
- X-axis:** Date
- Y-axis:** Bid Price, Offer Price, and Reference Price.

Your Task

You can reuse most of your plotting code from Challenge 1. Follow these steps:

**Choose a Ticker:** Pick one asset you traded to focus your analysis on.

**Filter the Data:** Create new lists for dates and prices by looping through your `market_maker.quoted_trades` log and selecting only the data for your chosen ticker.

**Initialize and Plot:** Use `plt.subplots()` to create your axes and `axes.plot()` to draw the three price series.

**Add Labels:** Complete your chart with a title (mentioning the ticker), axis labels, and a legend to make it readable.

```

[41]: # Insert your answer code here
import matplotlib.pyplot as plt

ticker_to_plot = "AAPL"
quote_dates = []
bid_data = []
offer_data = []
ref_data = []

for trade in quoted_trades:
    if trade.ticker == ticker_to_plot:

```

```

quote_dates.append(trade.date)
bid_data.append(trade.bid_price)
offer_data.append(trade.offer_price)
ref_data.append(trade.ref_price)

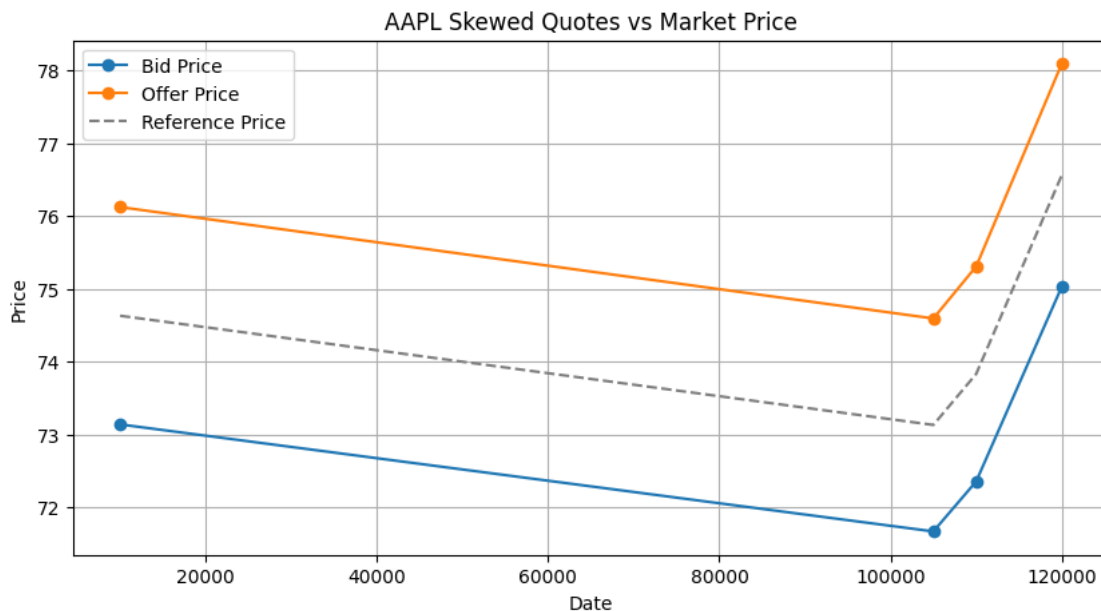
fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(quote_dates, bid_data, label="Bid Price", marker="o")
ax.plot(quote_dates, offer_data, label="Offer Price", marker="o")
ax.plot(quote_dates, ref_data, label="Reference Price", linestyle="--",
        color="gray")

ax.set_title(f"{ticker_to_plot} Skewed Quotes vs Market Price")
ax.set_xlabel("Date")
ax.set_ylabel("Price")
ax.legend()
ax.grid(True)

plt.show()

```



## 0.4 Beta Research & Hedging Strategy

Welcome to the research section. In this part of the assessment, your goal is to calculate the **beta** of five individual stocks relative to a FAANG ETF. Beta is a fundamental measure of a stock's volatility in relation to the broader market. The values you calculate here will be essential for building an optimized hedging strategy in Challenge 3.

#### Understanding Beta ( )

Beta tells you how much an asset's price is expected to move when the overall market moves.

<ul>

<li>A beta of <strong>1</strong> means the stock tends to move in line with the market.</li>

<li>A beta <strong>greater than 1</strong> means the stock is more volatile than the market.</li>

<li>A beta <strong>less than 1</strong> means the stock is less volatile than the market.</li>

</ul>

#### Formula for Beta

To find the beta of a stock, you'll use the covariance of its returns with the market's returns.

$$\beta_{\text{stock}} = \frac{\text{Cov}(R_{\text{stock}}, R_{\text{market}})}{\text{Var}(R_{\text{market}})}$$

Where  $R_{\text{stock}}$  represents the returns of the individual stock and  $R_{\text{market}}$  represents the market returns.

#### Your Task

Your objective is to process historical price data to calculate the daily returns for each of the five stocks and the FAANG ETF. You will then use these returns to calculate a beta value for each individual stock.

To start, let's import the packages we will use in this notebook.

```
[42]: # The code in this cell is used to import the packages to be used throughout
      ↪ this notebook.
      # The following are private packages available only during this simulation:
      from AmplifyQuantTrading import Data
      # The following are publicly available packages:
      from matplotlib import pyplot as plt
      import pandas
```

#### 0.4.1 How the FAANG ETF is Calculated

The FAANG ETF is an exchange-traded product composed of five equally-weighted stocks: Facebook, Amazon, Apple, Netflix, and Google.

#### Calculation Method

<ol>

<li><strong>Rebasing:</strong> At the start of the price series ( $t_0$ ), each of the 5 stock prices is rebased to 1.

<li><strong>Daily Changes:</strong> On any subsequent day ( $t_1$ ), the value of each rebased stock is calculated as the daily percentage change from the previous day.

<li><strong>Final ETF Value:</strong> The ETF's value on any given day is the simple average of the five rebased stock values.

</ol>

#### 0.4.2 The Problem with a Simple Hedge

In the previous simulation, you hedged your exposure by taking an equal and opposite position in the ETF (e.g., selling \$1m of the ETF when you were long \$1m of a stock). This is called a **nominal hedge**.

#### The Volatility Mismatch

A nominal hedge is inaccurate because individual stocks are often more or less volatile than the market.

To create a correct hedge, we must use **Beta ( )** to measure the relative volatility of a stock to the market.

### 0.4.3 Research Task: Calculate Hedging Ratios

Your goal is to use historical price data to find the precise beta for each stock relative to the ETF. This beta value will be your correct hedging ratio for Challenge 3.

Your Task

Load the provided 6 months of price data.

Calculate the daily percentage returns for each of the 5 stocks and the FAANG ETF.

For each stock, calculate the covariance between its returns and the ETF's returns.

Calculate the variance of the ETF's returns.

Apply the beta formula (

$$\beta = \frac{\text{Covariance}}{\text{Variance}}$$

) to find the precise hedging ratio for each of the 5 stocks.

```
[43]: # The code in this cell assigns the Data.  
      ↪get_price_series("PricestoFeedserverEtfUpdate") to the prices variable.  
prices = Data.get_price_series("PricestoFeedserverEtfUpdate")
```

```
[44]: prices_FB = []  
prices_AAPL = []  
prices_AMZN = []  
prices_NFLX = []  
prices_GOOGL = []  
prices_FAANG = []  
  
for price in prices:  
    if price[0] == "AAPL":  
        prices_AAPL.append(price)  
    elif price[0] == "FB":  
        prices_FB.append(price)  
    elif price[0] == "GOOGL":  
        prices_GOOGL.append(price)  
    elif price[0] == "AMZN":  
        prices_AMZN.append(price)  
    elif price[0] == "NFLX":  
        prices_NFLX.append(price)  
    elif price[0] == "FAANG":  
        prices_FAANG.append(price)  
  
AAPL_change = pandas.DataFrame(prices_AAPL)[2].pct_change()[1:]  
FB_change = pandas.DataFrame(prices_FB)[2].pct_change()[1:]  
GOOGL_change = pandas.DataFrame(prices_GOOGL)[2].pct_change()[1:]  
AMZN_change = pandas.DataFrame(prices_AMZN)[2].pct_change()[1:]  
NFLX_change = pandas.DataFrame(prices_NFLX)[2].pct_change()[1:]  
FAANG_change = pandas.DataFrame(prices_FAANG)[2].pct_change()[1:]
```

```
[49]: # Insert your answer code here

aapl = AAPL_change.reset_index(drop=True)
fb = FB_change.reset_index(drop=True)
goog = GOOGL_change.reset_index(drop=True)
amzn = AMZN_change.reset_index(drop=True)
nflx = NFLX_change.reset_index(drop=True)
fa = FAANG_change.reset_index(drop=True)

def align(x, y):
    m = min(len(x), len(y))
    return x.iloc[:m], y.iloc[:m]

betas = {}
for ticker, s in {
    "AAPL": aapl, "FB": fb, "GOOGL": goog, "AMZN": amzn, "NFLX": nflx
}.items():
    s_aligned, fa_aligned = align(s, fa)
    s_aligned = s_aligned[mask]
    fa_aligned = fa_aligned[mask]

    fa_var = fa_aligned.var()
    betas[ticker] = float("nan") if (fa_var == 0 or pandas.isna(fa_var)) else float(s_aligned.cov(fa_aligned) / fa_var)

beta_df = pandas.DataFrame.from_dict(betas, orient="index",
    columns=["beta_vs_FAANG"])
print(beta_df)
```

```

      beta_vs_FAANG
AAPL      0.864936
FB        1.032088
GOOGL     1.268621
AMZN      0.919466
NFLX      0.921326
```

```
[55]: # The code below creates the MarketMaker Object
mm = MarketMaker.mm()
prices = Data.get_price_series("PricestoFeedserverEtfUpdate", True)
price_requests = Data.get_price_requests("PriceRequeststoFeedserverExtended")
```

## 0.5 Challenge 3: Optimized ETF Hedging

Welcome to the final challenge. You will now upgrade your algorithm from Challenge 2 to perform a more sophisticated, **beta-adjusted hedge**. Instead of a simple \$1-for-\$1 hedge, you will use the beta values from your research to calculate the precise amount of ETF needed to offset your risk in each stock.

#### The Beta-Adjusted Hedge

The core of this challenge is to use beta as your hedging ratio. The formula is simple:

Hedge Amount = Equity Risk Amount × Beta

For example, if you complete a trade that leaves you with a **\\$1,000,000 long**

##### 0.5.1 New Tools for Hedging

To execute your hedge, you will interact with a simulated exchange. This requires a new object and two new methods.

#### Class Blueprint: `ExchangeTrade`

This object is a simple container used to send your ETF hedge order to the exchange.

- `ticker`: String (e.g., 'FAANG')
- `trade_volume`: Integer
- `ref_price`: Float
- `action`: String ('Buy' or 'Sell')
- `date`: Integer



#### New Methods for Hedging

**1. Execute a Hedge Trade**

- `Exchange.execute(trade)`: Sends your `ExchangeTrade` object to the exchange.

**2. Log Your ETF Position**

- `market_maker.update ETF_position(executed_trade)`: Logs the executed hedge

#### Data Format: Pandas DataFrame

For this section, all historical price data is stored in a **Pandas DataFrame**

##### 0.5.2 Challenge 3 - Task A: Implement the Full Hedging Loop

Your task is to modify your algorithm from Challenge 2 to include the beta-adjusted hedge after every completed trade.

Your Task

Copy your code from Challenge 2. Inside your main loop that processes each price request, add the new hedging steps after a trade is successfully completed.

Process a client request (check risk, skew prices, create and log quote, show to client).

Check the client's response.

If the trade is NOT refused:

Log the completed trade using `market_maker.add_trade()`.

— Start New Hedging Logic —

Calculate the required ETF hedge amount using the stock's Beta.

Determine the hedge direction (if you bought the stock, you must sell the ETF, and vice-versa).



Create an ExchangeTrade object for your ETF hedge.

Execute the hedge using Exchange.execute().

Log the result of your hedge using market\_maker.update ETF\_position().

</li>

```
[71]: class ExchangeTrade:
    def __init__(self, ticker, trade_volume, ref_price, action, date):
        self.ticker = ticker
        self.trade_volume = trade_volume
        self.ref_price = ref_price
        self.action = action
        self.date = date

[ ]:

[76]: # Insert your answer code here

# (ticker, date) -> ref price
ref_by_key = {}
for dt, row in prices.iterrows():
    dt_int = int(dt)
    for tkr, px in row.items():
        ref_by_key[(tkr, dt_int)] = float(px)

# FAANG by date for hedge sizing
if "FAANG" not in prices.columns:
    raise ValueError("FAANG column missing from prices; cannot hedge.")
faang_by_date = {int(idx): float(px) for idx, px in prices["FAANG"].items()}

def get_ref(ticker, date):
    return ref_by_key.get((ticker, int(date)))

def get_beta(ticker):
    try:
        return float(betas.get(ticker, 1.0))
    except NameError:
        return 1.0

# quoting parameters
base_spread_pct = 0.02
skew_abs_pct = 0.01
ETF_TICKER = "FAANG"

date_index_set = set(int(x) for x in prices.index)

for req in price_requests:
```

```

if len(req) == 3:
    tkr, a, b = req
    if isinstance(a, (int, float)) and int(a) in date_index_set:
        date, trade_volume = int(a), int(b)
    elif isinstance(b, (int, float)) and int(b) in date_index_set:
        date, trade_volume = int(b), int(a)
    else:
        date, trade_volume = int(a), int(b)
    ticker = tkr
elif len(req) == 2:
    ticker, date = req
    date = int(date)
    trade_volume = 0
else:
    continue

ref_price = get_ref(ticker, date)
if ref_price is None:
    continue

# inventory-based skew
pos_obj = mm.current_positions.get(ticker, 0)
pos_val = getattr(pos_obj, "volume", pos_obj)
if isinstance(pos_val, (int, float)):
    if pos_val > 0: # long -> skew down to encourage selling
        skew = -skew_abs_pct
    elif pos_val < 0: # short -> skew up to encourage buying
        skew = +skew_abs_pct
    else:
        skew = 0.0
else:
    skew = 0.0

bid_price = ref_price * (1 - base_spread_pct + skew)
offer_price = ref_price * (1 + base_spread_pct + skew)

# create and log the quote, and show to client
quote = QuotedTrade(
    ticker=ticker,
    trade_volume=trade_volume,
    ref_price=ref_price,
    bid_price=bid_price,
    offer_price=offer_price,
    date=date
)
mm.add_quoted_trade(quote)
response = hf.show(quote)

```

```

side = str(response.hf_action).lower()
if side in ("buy", "sell"):
    # log completed trade
    completed = CompletedTrade(
        ticker=response.ticker,
        trade_volume=response.trade_volume,
        trade_price=response.trade_price,
        mm_action=response.hf_action,
        ref_price=response.ref_price,
        bid_price=response.bid_price,
        offer_price=response.offer_price,
        date=response.date
    )
    mm.add_trade(completed)

    # New Hedging Logic
    beta = get_beta(completed.ticker)

    # HF sells to us -> we BUY stock -> +exposure
    # HF buys from us -> we SELL stock -> -exposure
    exposure_sign = +1 if side == "sell" else -1
    equity_exposure = exposure_sign * float(completed.trade_price) *
    ↪int(completed.trade_volume)

    hedge_dollars = equity_exposure * beta
    etf_action = "Sell" if hedge_dollars > 0 else "Buy"

    etf_ref = faang_by_date.get(int(completed.date))
    if etf_ref and etf_ref > 0:
        etf_shares = int(round(abs(hedge_dollars) / etf_ref))
        if etf_shares > 0:
            etf_order = ExchangeTrade(
                ticker=ETF_TICKER,
                trade_volume=etf_shares,
                ref_price=etf_ref,
                action=etf_action,
                date=int(completed.date)
            )
            executed = Exchange.execute(etf_order)
            mm.update ETF_position(executed)

```

[77]: # GRADING CELL

## 0.6 Challenge 3 - Task B: Visualize Your Total Risk Exposure

For your final analysis, you will create a chart showing the **combined nominal risk** of your entire trading book over time. This graph is crucial as it shows how well your hedging strategy managed to keep your net risk under control. A successful market maker aims to keep this value from growing too large in either a long or short direction.

<h4>Chart Requirements</h4>

<p>This plot will track the net value of all open positions over the entire simulation.</p>

<ul>  
    <li><strong>Data Source:</strong> The balance history, which tracks the total nominal value

    <li><strong>X-axis:</strong> Time / Date</li>

    <li><strong>Y-axis:</strong> Combined Nominal Risk Value (\$)</li>

</ul>

Your Task

Extract the Data: Create lists for time and the corresponding total risk value from the simulation's balance log.

Initialize and Plot: Use `plt.subplots()` to create your axes and `axes.plot()` to draw the risk value over time.

Add Labels and a Zero Line: Give your chart a clear title (e.g., "Total Net Position Over Time") and axis labels. It is highly recommended to add a horizontal line at  $y=0$  to clearly show the moments you are net long (above the line) or net short (below the line).

```
[78]: # Insert your answer code here

history_candidates = [
    "balance_history",
    "net_exposure_history",
    "position_value_history",
]
mm_history = None
for name in history_candidates:
    if hasattr(mm, name):
        mm_history = getattr(mm, name)
        break

if mm_history is not None and len(mm_history) > 0:
    if isinstance(mm_history, dict):
        ts = sorted((int(k), float(v)) for k, v in mm_history.items())
    else:
        ts = []
        for rec in mm_history:
            if isinstance(rec, (list, tuple)) and len(rec) >= 2:
                ts.append((int(rec[0]), float(rec[1])))
            else:
                # try attributes date/value
```

```

        dt = int(getattr(rec, "date"))
        val = float(getattr(rec, "value"))
        ts.append((dt, val))
time_vals = [t for t, _ in ts]
risk_vals = [v for _, v in ts]

else:
    # reconstruct net nominal exposure over time
    # prices: wide DF, columns=tickers, index=dates
    # price lookup (ticker, date) -> price
    ref_by_key = {}
    for dt, row in prices.iterrows():
        dt = int(dt)
        for tkr, px in row.items():
            ref_by_key[(tkr, dt)] = float(px)

    def ref_price_at(ticker, date):
        return ref_by_key.get((ticker, int(date)))

    # collect trade events: completed cash equity trades + ETF hedge trades
    events = []
    for tr in getattr(mm, "completed_trades", []):
        tkr = tr.ticker
        dt = int(tr.date)
        vol = int(tr.trade_volume)
        # signed volume
        mm_side = str(tr.mm_action).lower()
        signed_vol = vol if mm_side == "buy" else -vol
        events.append((dt, tkr, signed_vol))

    # ETF hedge trades
    etf_log = getattr(mm, "ETF_positions", None)
    if etf_log is not None:
        for tr in etf_log:
            tkr = getattr(tr, "ticker", "FAANG")
            dt = int(tr.date)
            vol = int(getattr(tr, "trade_volume", 0))
            act = str(getattr(tr, "action", "Buy")).lower()
            signed_vol = vol if act == "buy" else -vol
            events.append((dt, tkr, signed_vol))

    events.sort(key=lambda x: x[0])
    positions = {} # ticker -> shares
    time_vals = []
    risk_vals = []

    for dt, tkr, dvol in events:

```

```

positions[tkr] = positions.get(tkr, 0) + dvol

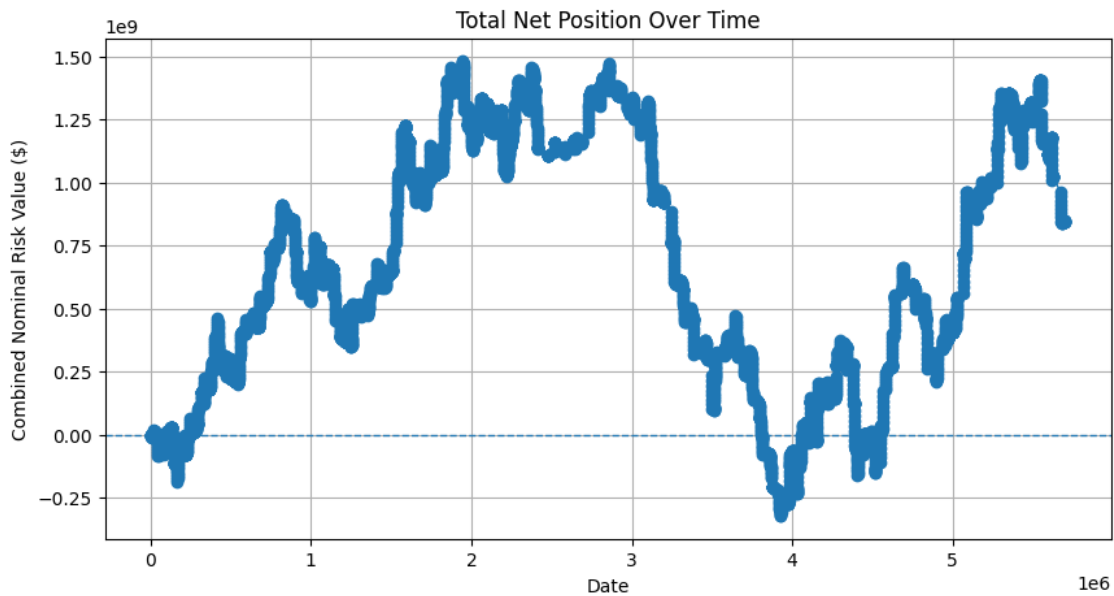
# compute net notional value
net_val = 0.0
# value each open position at current ref price
for tkr, sh in positions.items():
    px = ref_price_at(tkr, dt)
    if px is not None:
        net_val += sh * px

time_vals.append(dt)
risk_vals.append(net_val)

# plot
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(time_vals, risk_vals, marker="o", linewidth=1)
ax.axhline(0, linewidth=1, linestyle="--")

ax.set_title("Total Net Position Over Time")
ax.set_xlabel("Date")
ax.set_ylabel("Combined Nominal Risk Value ($)")
ax.grid(True)
plt.show()

```



## 0.7 Arbitrage Trading Research

This research section introduces **arbitrage**, a strategy that seeks to profit from temporary price differences between highly correlated assets. Your goal is to find and exploit these price inefficiencies between the real FAANG ETF and its five underlying component stocks.

While the ETF should theoretically mirror the average price changes of its components, market friction can cause them to diverge. Your challenge is to build an algorithm that can spot these moments and capitalize on them.

### The Arbitrage Strategy

The core of this strategy is to compare the price of the actual ETF with a "fair value" that

- Real ETF:** The traded price of the FAANG ETF from the dataset.
- Synthetic ETF:** A "fair value" you calculate by taking the average of the

The trading logic is based on which one is cheaper:

- If **Real ETF < Synthetic ETF** → Buy the cheap Real ETF and sell the expensive Synthetic ETF
- If **Real ETF > Synthetic ETF** → Sell the expensive Real ETF and buy the cheap Synthetic ETF

Profit is realized when the two prices converge back to their fair relationship.

## 0.8 Research Task: Identify Arbitrage Opportunities

Your task is to analyze 6 months of historical price data to build a model that flags potential arbitrage trades.

Your Task

Load the provided price data into a Pandas DataFrame.

Calculate the value of the Synthetic ETF for each day based on the average price movements of the 5 component stocks.

Create a new column or series representing the spread (i.e., the difference) between the Real ETF and your Synthetic ETF.

Identify the points in time where this spread is significantly different from zero. These are your potential arbitrage opportunities.

```
[80]: # The code in this cell assigns the Data.  
      ↪get_price_series("PricestoFeedserverEtfUpdate") to the prices variable.  
prices = Data.get_price_series("PricestoFeedserverEtfUpdate")
```

```
[81]: prices_FB = []  
      prices_AAPL = []  
      prices_AMZN = []  
      prices_NFLX = []  
      prices_GOOGL = []  
      prices_FAANG = []
```

```

for price in prices:
    if price[0] == "AAPL":
        prices_AAPL.append(price)
    elif price[0] == "FB":
        prices_FB.append(price)
    elif price[0] == "GOOGL":
        prices_GOOGL.append(price)
    elif price[0] == "AMZN":
        prices_AMZN.append(price)
    elif price[0] == "NFLX":
        prices_NFLX.append(price)
    elif price[0] == "FAANG":
        prices_FAANG.append(price)

FB_df = pandas.DataFrame(prices_FB)[2]
AAPL_df = pandas.DataFrame(prices_AAPL)[2]
AMZN_df = pandas.DataFrame(prices_AMZN)[2]
NFLX_df = pandas.DataFrame(prices_NFLX)[2]
GOOGL_df = pandas.DataFrame(prices_GOOGL)[2]
FAANG_df = pandas.DataFrame(prices_FAANG)[2]

```

```

[82]: # Insert your answer code here
df = pd.concat(
    [FB_df.rename("FB"), AAPL_df.rename("AAPL"), AMZN_df.rename("AMZN"),
     NFLX_df.rename("NFLX"), GOOGL_df.rename("GOOGL"), FAANG_df.
     ↪rename("FAANG")],
    axis=1
).dropna()

# calculate synthetic ETF
components = ["FB", "AAPL", "AMZN", "NFLX", "GOOGL"]
comp_rets = df[components].pct_change().fillna(0.0)
synthetic_ret = comp_rets.mean(axis=1)

synthetic_price = df["FAANG"].iloc[0] * (1 + synthetic_ret).cumprod()
synthetic_price.name = "Synthetic ETF"

# calculate spread
spread = (df["FAANG"] - synthetic_price).rename("Spread")

# identify arbitrage opportunities
spread_z = ((spread - spread.mean()) / spread.std(ddof=0)).rename("Spread_z")
arbitrage_opps = pd.DataFrame({
    "FAANG_real": df["FAANG"],
    "Synthetic ETF": synthetic_price,
    "Spread": spread,
    "Spread_z": spread_z
})

```



```

})
arbitrage_opps = arbitrage_opps[arbitrage_opps["Spread_z"].abs() >= 2.0]

display(arbitrage_opps.head())

```

|    | FAANG_real | Synthetic ETF | Spread   | Spread_z |
|----|------------|---------------|----------|----------|
| 47 | 94.50      | 89.688471     | 4.811529 | 2.183039 |
| 48 | 93.80      | 89.726763     | 4.073237 | 2.058391 |
| 63 | 94.90      | 90.826115     | 4.073885 | 2.058501 |
| 71 | 90.12      | 82.753455     | 7.366545 | 2.614410 |
| 72 | 89.10      | 82.925980     | 6.174020 | 2.413073 |

<h4>Visualize Your Findings</h4>

<p>This space is for you to create graphs that showcase your research findings. A strong visual

<p><strong>Suggestions for your plot:</strong></p>

<ul>

<li>Plot the price of the <strong>Real ETF</strong> and your <strong>Synthetic ETF</strong>

<li>In a separate subplot below, plot the <strong>spread</strong> (the difference between t

</ul>

[83]: *# Insert your answer code here*

```

# align series on the same index
real = df["FAANG"]
synth = synthetic_price
common_idx = real.index.intersection(synth.index)
real = real.loc[common_idx]
synth = synth.loc[common_idx]
spread_plot = (real - synth)

# recompute z and pick signals (|z| >= 2)
mu = spread_plot.mean()
sigma = spread_plot.std(ddof=0)
z = (spread_plot - mu) / sigma
signals_idx = z.index[(z.abs() >= 2.0)]
signals_vals = spread_plot.loc[signals_idx]

fig, axes = plt.subplots(2, 1, figsize=(11, 7), sharex=True,
    ↪ gridspec_kw={"height_ratios": [2, 1]})

axes[0].plot(common_idx, real, label="FAANG (Real)")
axes[0].plot(common_idx, synth, label="Synthetic ETF")
axes[0].set_title("Real FAANG vs Synthetic ETF (Last ~6 Months)")
axes[0].set_ylabel("Price")
axes[0].legend()
axes[0].grid(True)

axes[1].plot(common_idx, spread_plot, label="Spread (Real - Synthetic)")

```

```

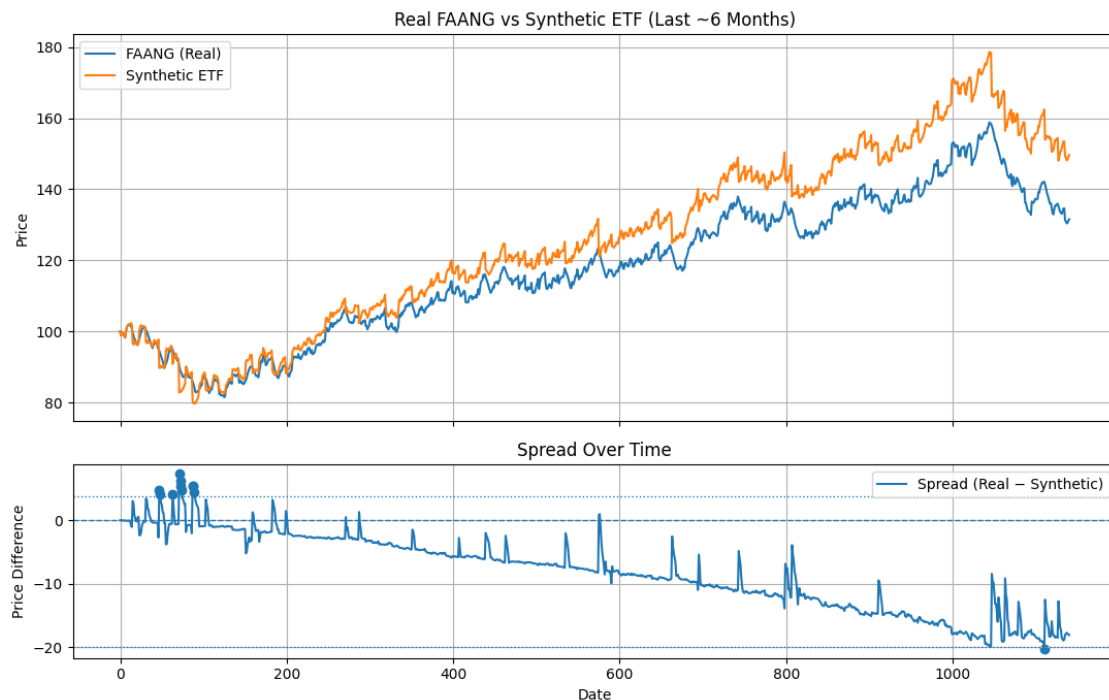
axes[1].axhline(0, linestyle="--", linewidth=1)
axes[1].axhline(mu + 2*sigma, linestyle=":", linewidth=1)
axes[1].axhline(mu - 2*sigma, linestyle=":", linewidth=1)

# signal points
if len(signals_idx) > 0:
    axes[1].scatter(signals_idx, signals_vals)

axes[1].set_title("Spread Over Time")
axes[1].set_xlabel("Date")
axes[1].set_ylabel("Price Difference")
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()

```



## 0.9 Challenge 4: Arbitrage Trading

Welcome to the final challenge, which introduces **arbitrage**: a strategy that exploits temporary price differences between highly correlated assets. Your goal is to find and capitalize on these inefficiencies between the real FAANG ETF and its five underlying component stocks.

While the ETF should theoretically mirror its components, market friction can cause them to diverge. You will build an algorithm that detects these moments and executes trades to profit from

the eventual price convergence.

#### The Arbitrage Strategy

The core of this strategy is to compare the traded price of the ETF with a "fair value" that



- Real ETF:** The actual, traded price of the FAANG ETF from the dataset.

- Synthetic ETF:** A "fair value" you calculate based on the average price



The trading logic is based on which one is cheaper:



- If **Real ETF > Synthetic ETF** → Sell the expensive Real ETF and buy the

- If **Real ETF < Synthetic ETF** → Buy the cheap Real ETF and sell the exp



### 0.9.1 Trading Tools: The HedgeFund Object

For this challenge, you will use a HedgeFund object (`hf`) to manage your account and execute trades.

#### The `hf` Class

This is your main controller for executing trades. Its key properties and methods are:



- `.balance`: Your current account balance in dollars.

- `.current_positions`: A dictionary containing `CurrentPosition`

- `.commission_percentage`: The commission fee for each trade.



---

**Method:** `hf.execute_order(ticker, volume, action, date)`

This function sends a trade order to the market. It requires the ticker, volume, action ('BUY'

#### The `CurrentPosition` Class

This object, found within the main `hf` object, tracks and manages all details re

---

**Key Properties:**



- `ticker` (str): The symbol of the asset (e.g., 'AAPL').

- `direction` (str): Your current stance in the asset ('LONG', 'SHORT', or 'FL

- `position_volume` (int): The number of shares you currently hold.

- `open_price` (float): The average price at which you entered your current p

- `profit_loss` (float): The current unrealized profit or loss on the open pos

- `commission_costs` (float): The total commission fees paid for this asset's

- `trade_history` (list): A complete log of every transaction for this asset.



`<details>`

`<summary><strong>Click to see Trade History details</strong></summary>`

`<div style="background:#FFFFFF;padding:10px;border:1px solid #E9ECEF;border-radius:5px;margin`

`<p>The trade_history is a list of dictionaries, where each dictionary represents`

`<ul>`

- `action`: 'BUY' or 'SELL'.

- `trade_price`: The price of the individual transaction.

```

<li><code>trade_volume</code>: The volume of the individual transaction.</li>
<li><code>realized_profit_loss</code>: The P&L closed out by this specific trade.</li>
<li><code>commission_costs</code>: The commission fee for this specific trade.</li>
<li><code>date</code>: The date of the transaction.</li>
</ul>
</div>
</details>

```

## 0.9.2 Challenge 4 - Task A: Build the Arbitrage Algorithm

Your primary task is to create a function that identifies and acts on arbitrage opportunities using the provided data and trading tools.

Your Task

**Prepare Your Data:** Load the price data into a Pandas DataFrame. Calculate the value of the Synthetic ETF and the spread (difference) between the Real and Synthetic ETFs for each day.

**Develop a Trading Signal:** Loop through your data day-by-day. Your algorithm must decide when the spread is large enough to be a profitable trading opportunity (i.e., it must be wider than the transaction costs).

**Execute the Arbitrage:** When your signal is triggered, use the `hf.execute_order()` method to simultaneously place all 6 trades (1 for the ETF and 1 for each of the 5 stocks) in the correct directions.

**Manage the Position:** You must also implement logic to close out your positions (by executing the opposite trades) once the spread has converged back to zero.

```

[106]: # The code in this cell is used to import the packages to be used throughout
        ↪this notebook.
        # The following are private packages available only during this simulation:
        from AmplifyQuantTrading import Data
        # The following are publicly available packages:
        from matplotlib import pyplot as plt
        import pandas
        # The code in this cell is used to import the packages to be used throughout
        ↪this task.
        from AmplifyArbitrageTrading import HedgeFund
        # The code in this cell assigns the Data.
        ↪get_price_series("PricestoFeedserverEtfUpdate", True) to the prices variable.
        prices = Data.get_price_series("PricestoFeedserverEtfUpdate", True)

```

```

[107]: # The code below creates the HedgeFund object
        hedge_fund = HedgeFund.hf(commission_percentage=0.001, data=prices)

```

```

[108]: # Insert your answer code here

        # prepare data
        if isinstance(prices, pd.DataFrame):

```

```

    df = prices.copy().sort_index()
else:
    tmp = pd.DataFrame(prices, columns=["ticker", "date", "price"])
    df = tmp.pivot(index="date", columns="ticker", values="price").sort_index()

components = ["FB", "AAPL", "AMZN", "NFLX", "GOOGL"]
assert "FAANG" in df.columns, "FAANG price series missing"
assert all(c in df.columns for c in components), "One or more component series_
missing"

window = min(126, len(df))
df = df.tail(window)

# Synthetic ETF
comp_rets = df[components].pct_change().fillna(0.0)
synthetic_ret = comp_rets.mean(axis=1)
synthetic_price = df["FAANG"].iloc[0] * (1 + synthetic_ret).cumprod()

# Spread = Real - Synthetic
spread = df["FAANG"] - synthetic_price

def compute_share_pack(date_idx):
    etf_px = float(df.loc[date_idx, "FAANG"])
    comp_px = df.loc[date_idx, components].astype(float)
    base_shares = (etf_px / 5.0) / comp_px
    scale = 10
    comp_shares = (base_shares * scale).round().astype(int).clip(lower=1)
    basket_notional = float((comp_shares * comp_px).sum())
    etf_shares = max(1, int(round(basket_notional / etf_px)))
    return etf_shares, comp_shares.to_dict()

# entry threshold: must exceed estimated transaction cost
def entry_threshold(date_idx):
    etf_px = float(df.loc[date_idx, "FAANG"])
    comp_sum = float(df.loc[date_idx, components].sum())
    notional = etf_px + comp_sum
    return 2.0 * hedge_fund.commission_percentage * notional

position_open = False
open_side = None
open_lots = None
entry_sign = 0

Z_ENTER = 2.0
Z_EXIT = 0.5

for date_idx in df.index:

```

```

spr = float(spread.loc[date_idx])
z    = float(spread_z.loc[date_idx])

etf_qty, comp_qtys = compute_share_pack(date_idx)
basket_notional = float(df.loc[date_idx, "FAANG"]) * etf_qty \
    + float((pandas.Series(comp_qtys) * df.loc[date_idx,
↪components])).sum())
break_even = 2.0 * hedge_fund.commission_percentage * basket_notional
spread_dollars = abs(spr) * etf_qty

if not position_open:
    if (abs(z) >= Z_ENTER) and (spread_dollars > break_even):
        if spr > 0:
            # Real > Synthetic -> SELL ETF, BUY components
            hedge_fund.execute_order("FAANG", etf_qty, "SELL",
↪int(date_idx))
            for tkr, vol in comp_qtys.items():
                hedge_fund.execute_order(tkr, int(vol), "BUY",
↪int(date_idx))
            open_side, entry_sign = "short ETF", +1
        else:
            # Real < Synthetic -> BUY ETF, SELL components
            hedge_fund.execute_order("FAANG", etf_qty, "BUY", int(date_idx))
            for tkr, vol in comp_qtys.items():
                hedge_fund.execute_order(tkr, int(vol), "SELL",
↪int(date_idx))
            open_side, entry_sign = "long ETF", -1

        open_lots = {"FAANG": etf_qty, **{k: int(v) for k, v in comp_qtys.
↪items()}}
        position_open = True

    else:
        # exit when spread reverts or changes sign
        if (abs(z) <= Z_EXIT) or (spr * entry_sign <= 0):
            if open_side == "short ETF":
                hedge_fund.execute_order("FAANG", open_lots["FAANG"], "BUY",
↪int(date_idx))
                for tkr in components:
                    hedge_fund.execute_order(tkr, open_lots[tkr], "SELL",
↪int(date_idx))
            else:
                hedge_fund.execute_order("FAANG", open_lots["FAANG"], "SELL",
↪int(date_idx))
                for tkr in components:

```

```

        hedge_fund.execute_order(tkr, open_lots[tkr], "BUY",
↪int(date_idx))

        position_open = False
        open_side = None
        open_lots = None
        entry_sign = 0

```

```

[104]: pnl = 0
for ticker in prices:
    pnl += hedge_fund.current_positions[ticker].profit_loss - hedge_fund.
↪current_positions[ticker].commission_costs
    # If you wanted to print the Dataframes showing each trade.
    # print(pandas.DataFrame.from_records(hedge_fund.current_positions[ticker].
↪trade_history))

print("Ending Balance: ", hedge_fund.balance)

print("Total Profit or Loss (including commission):", pnl)

```

Ending Balance: 20000676.16

Total Profit or Loss (including commission): 628.8299999999999

```

[109]: # GRADING CELL

```

```

[ ]:

```